

Day 5 — Backend Build, Dashboard Integration & Repository Governance

Date: 3 July 2026

1. Aim of Today's Work

Today had two threads running together. The first was making sure everything built so far is properly organised, version-controlled, and self-documenting going forward — the team's GitHub repository, branch protection, and an automated daily activity log were all set up and verified working. The second was real feature work: the backend was designed and built from scratch, and both dashboards were connected to it and extended with new features.

Hands-on ELM327 coding on a real car was deliberately not done today — that work is picked up in the next session, once the direction from this morning's meeting (see Section 2) is reflected in the plan.

2. Morning Meeting — Direction Given

Three points of direction came out of this morning's team call:

1. Proof-of-concept first. Before attempting Ackermann (car-style) steering or anything more elaborate, get a simple proof-of-concept vehicle working to demonstrate the concept — this matches the Phase 1 plan already in place.
2. Streaming via a central server. Instead of dashboards talking directly to hardware, route everything through a server with an API endpoint, backed by a proper database — the backend work in Section 3 is the team's response to this.
3. ELM327 done in-house, wired directly. Rather than relying on the ELM327's own Bluetooth app (which will not get sign-off, since everything needs to be in-house and integrated into the team's own ecosystem), the ELM327/OBD line should be connected directly to the ESP32 and CAN, and the data decoded by the team's own code.

3. Backend — Built and Deployed

The backend was designed and built today as the central hub between hardware and both dashboards. Previously the dashboards connected directly to the ESP32's own WebSocket — that only allowed one client at a time and kept no history. The new backend fixes both: every message is saved and broadcast to every connected client at once.

3.1 Architecture

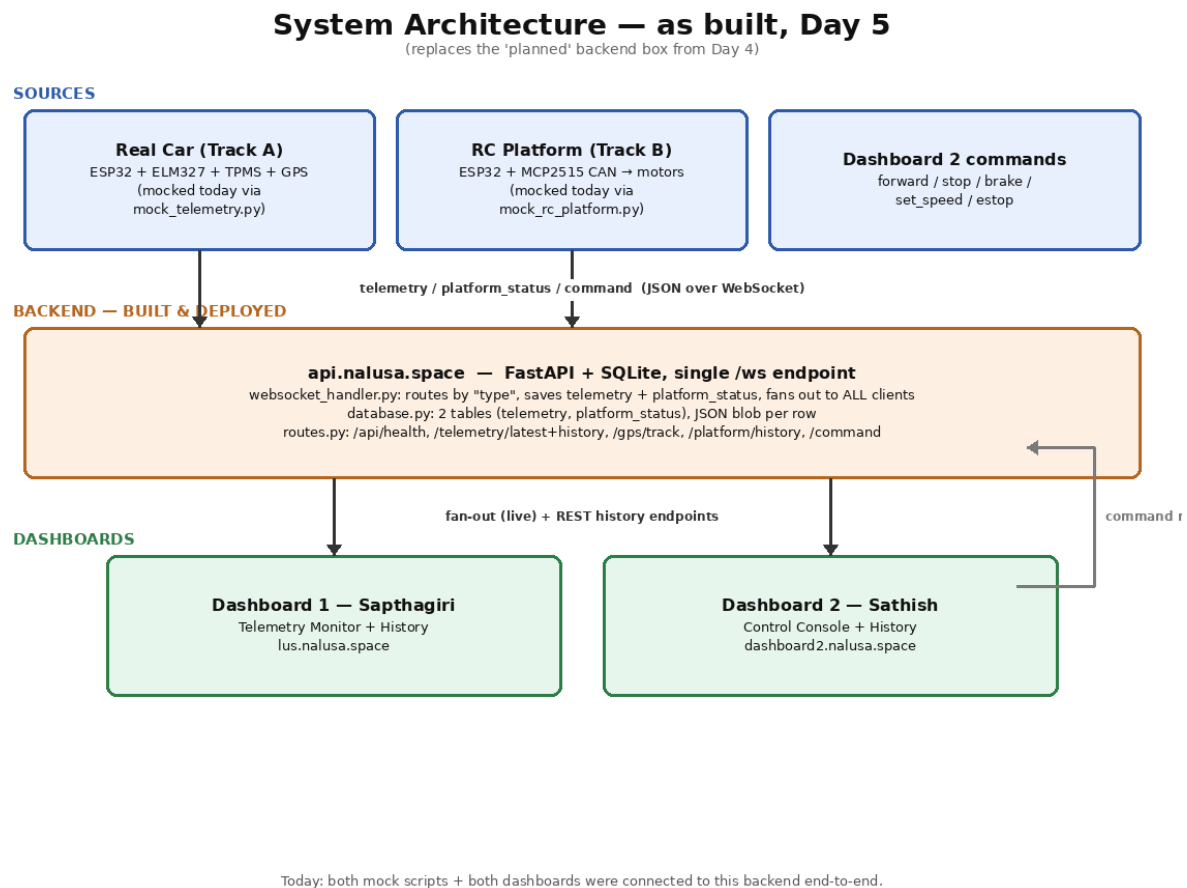


Figure: system architecture as built today — sources, backend, and both dashboards

3.2 Stack and Key Decisions

Decision	Choice	Why
Language / framework	Python + FastAPI	REST and WebSocket in one framework; auto-generated interactive API docs
Database	SQLite	Zero infrastructure to set up; a prototype-scale project doesn't need a database server; the file can be deleted and recreated instantly if needed
Storage format	JSON blob per row	If the message schema gains fields, no database migration is needed
WebSocket design	One single /ws endpoint for everyone	Every message already carries a type field per the schema, so one endpoint can route all message kinds without needing separate endpoints per client

3.3 File Structure

```
Car Backend/ ├── main.py          (FastAPI app entry point, CORS, startup) ├── config.py
(host, port, DB path) ├── database.py      (SQLite schema + read/write functions) ├──
websocket_handler.py (connection lifecycle + message routing + fan-out) ├── routes.py
(REST API endpoints) ├── mock_telemetry.py (test script - simulates the real car) ├──
mock_rc_platform.py  (test script - simulates the RC platform) └── telemetry.db
(SQLite database, auto-created on first run)
```

3.4 REST API Endpoints

Method	Endpoint	Returns
GET	/api/health	{"status": "ok"} — connectivity check used by both dashboards
GET	/api/telemetry/latest	Most recent telemetry record
GET	/api/telemetry/history?limit=100	Last N telemetry records, oldest first (for graphs)
GET	/api/gps/track?limit=200	Last N valid GPS points as {lat, lng, ts} — feeds the map trail
GET	/api/platform/history?limit=200	Last N platform_status records (for replay / history)
POST	/api/command	Fallback command route over HTTP if the WebSocket is unavailable

Base URL: <https://api.nalusa.space> · WebSocket: <wss://api.nalusa.space/ws> · Interactive API docs: </docs>

3.5 Message Routing

- type: "telemetry" → saved to the telemetry table and broadcast to every connected client
- type: "platform_status" → saved to the platform_status table and broadcast to every connected client
- type: "command" → NOT saved (commands are instantaneous control signals, not history) — relayed live to reach the hardware side

If a client disconnects without a clean close, the fan-out function catches the failed send and removes it from the connected-clients list automatically.

3.6 Mock Test Scripts

Two scripts simulate hardware so the whole system could be tested today without physical hardware present — the same “fake sender” approach planned back on Day 4.

- mock_telemetry.py — sends a full telemetry message every second with realistic, slightly randomised values; GPS drifts each tick to draw a trail on Dashboard 1's map. Auto-reconnects if the backend restarts.
- mock_rc_platform.py — sends platform_status at 5 messages/second and listens for and reacts to real command messages from Dashboard 2 (forward, set_speed, stop, brake, estop), including the estop-always-wins safety rule. Also simulates an obstacle occasionally triggering the SLOWING/BRAKING avoidance states.

3.7 Still Needed (from the backend report)

- Historical Data UI — the history REST endpoints are live; both dashboards need to call them and render the results (each dashboard already has a History button added today — see Section 4).
- Real hardware integration — the mock scripts need to be replaced with actual ESP32 firmware sending real messages to /ws, using the same MESSAGE_SCHEMA.md format already in place.
- Authentication — the WebSocket and REST endpoints are currently open with no API key or token; worth adding if this goes beyond the team's own network.

Open question to confirm with leadership

This morning's direction was to stream via a central server backed by a proper database, in the context of using the company's own cloud infrastructure and its RDBMS. What was actually built and deployed today is a self-hosted backend (FastAPI + SQLite) on the team's own domain (api.nalusa.space), not the company's cloud service.

The backend itself is solid and working end-to-end, so this is not a quality issue — it's worth explicitly confirming whether this self-hosted version satisfies the direction given, or whether the same backend design needs to be migrated onto the company's actual cloud server and database once those details are available.

4. Dashboard UI Updates

Both dashboards were connected to the new backend today and extended with new features. Screenshots below are from this evening's testing.

4.1 Dashboard 1 — Telemetry Monitor (Sapthagiri)

Now points at the backend (wss://api.nalusa.space/ws) instead of connecting directly to hardware. A History button and a “Load Past Route” control were added on the GPS panel, ready to call the new /api/gps/track and /api/telemetry/history endpoints.

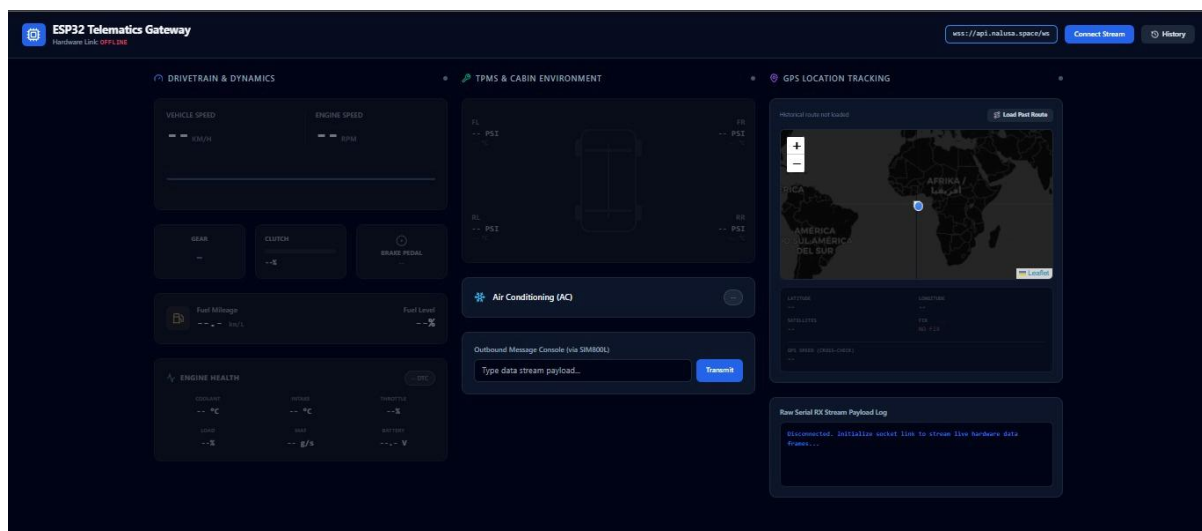


Figure: Dashboard 1 before connecting — note the History button (top right) and Load Past Route control on the map panel, both new today

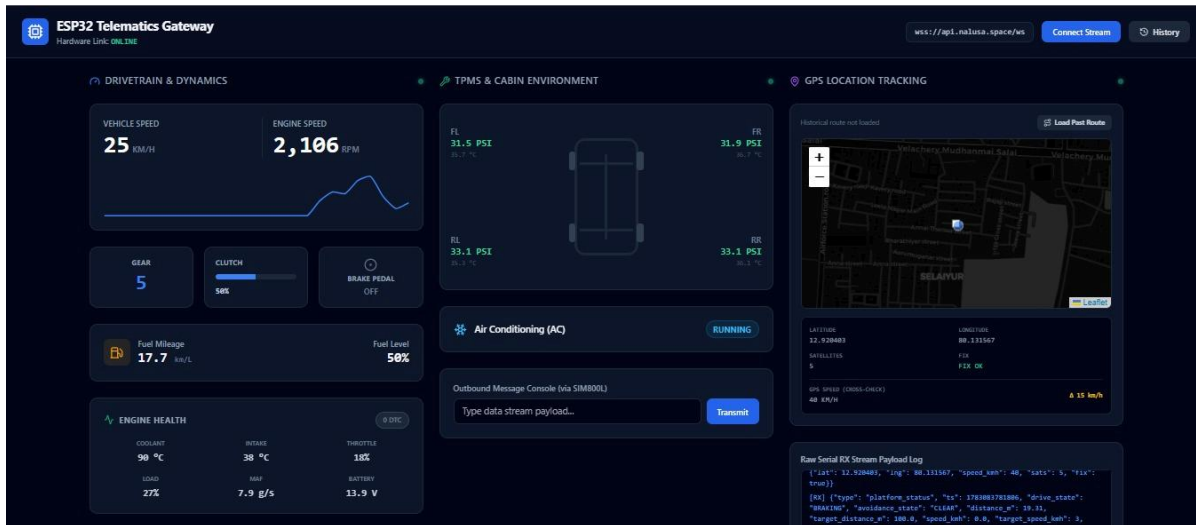


Figure: Dashboard 1 connected live to the backend — real-looking mock data flowing through: speed, RPM, tyre pressures, engine health, and a GPS fix with 5 satellites. The raw log at bottom right shows the actual JSON messages arriving, including platform_status frames the dashboard receives but does not render (by design — those are for Dashboard 2)

4.2 Dashboard 2 — Vehicle Control Console (Sathish)

Rebuilt with a fuller control layout — Forward, Backward, Left, Right, Brake, plus separate Start/Stop controls, a History button, and the existing target-speed slider and Emergency Stop. Connects to the same backend WebSocket.

Schema note — flagged correctly, not silently invented

The new Left / Right / Backward / Start buttons send action values that are not yet part of MESSAGE_SCHEMA.md (which currently defines forward, set_speed, stop, brake, estop only). This was called out directly in the UI itself ("pending addition to MESSAGE_SCHEMA.md") rather than being wired up silently.

Next step: agree the additional action names and any new fields (e.g. a turn direction or angle) with the team lead, add them to the schema, then wire these buttons to actually send commands.

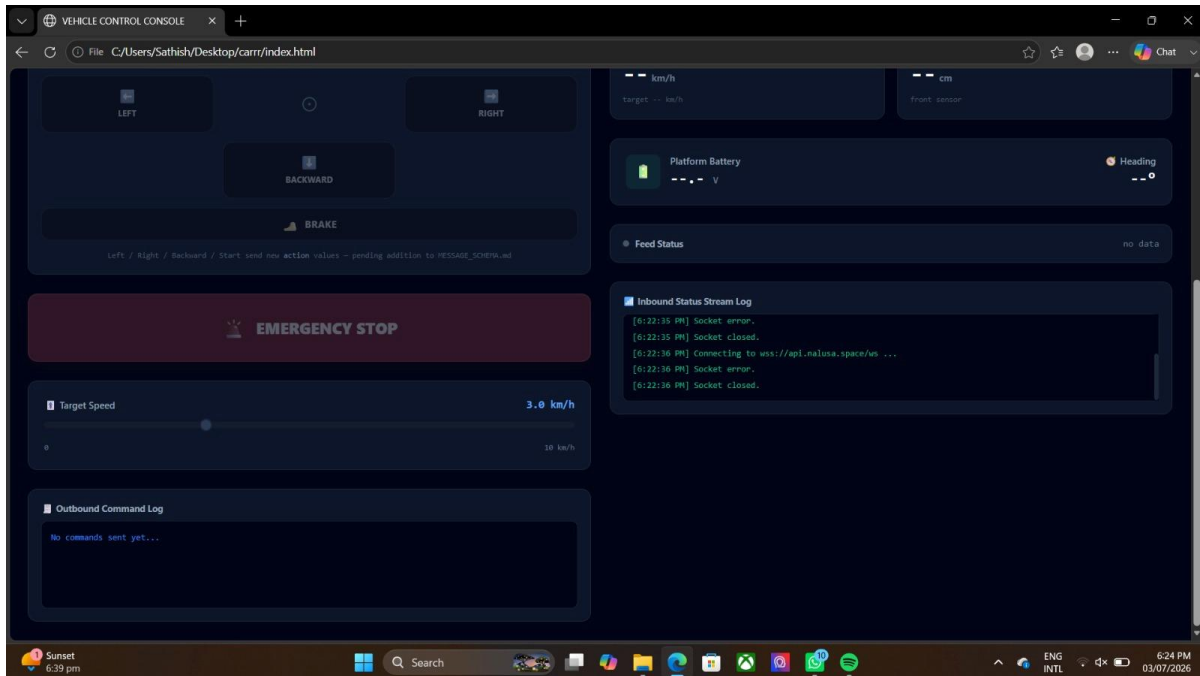


Figure: Dashboard 2's expanded drive controls — Left / Right / Backward added alongside Forward and Brake, with the schema-pending note visible beneath the button grid

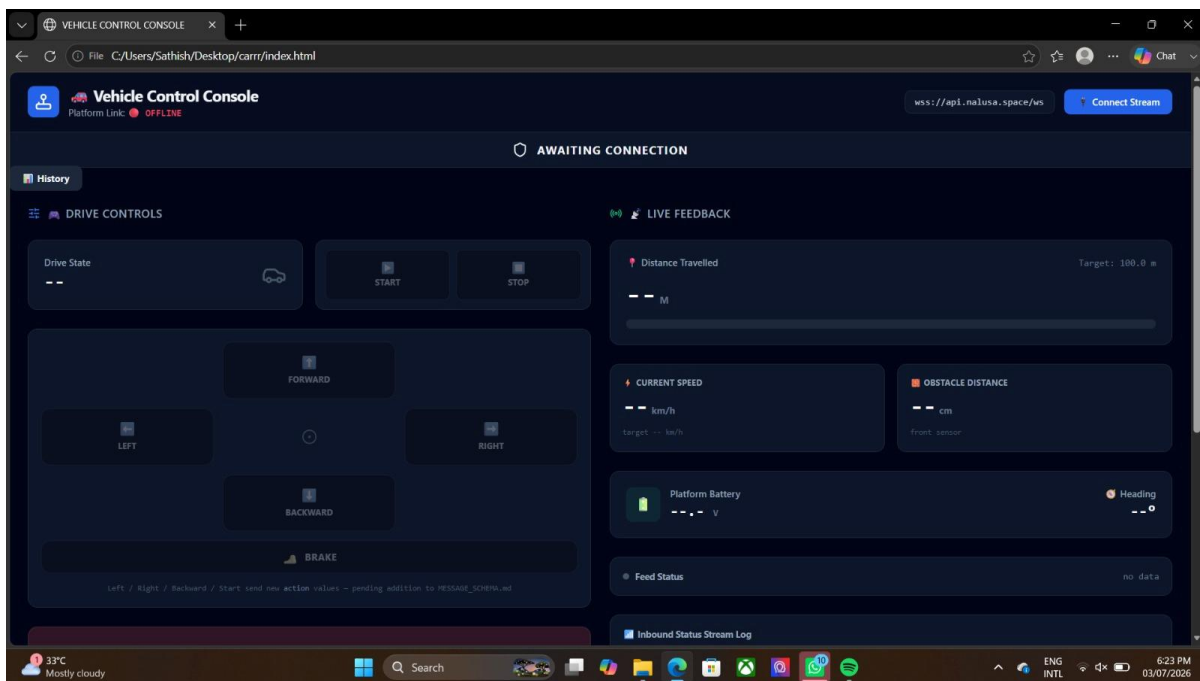


Figure: Dashboard 2 full layout — History button added top-left, Drive State readout with separate Start/Stop, awaiting-connection banner

5. GitHub Repository & Daily Documentation System

Alongside the feature work, the team's GitHub organisation and repository were fully set up today, so all work — code, docs, and day-to-day progress — lives in one governed place.

5.1 Organisation, Repository and Onboarding

- Created the GitHub organisation and the Car-Automation repository, with all five teammates invited.
- Repository structured with one folder per track (dashboard-telemetry, dashboard-control, backend, firmware, hardware, sensing, docs), a root README with a live status board, and a CONTRIBUTING guide setting the branch → pull request → review workflow.
- An ONBOARDING guide was written and shared so every teammate could set up Git, generate a personal access token, clone the repo, and push their own work independently.

5.2 Branch Protection

A ruleset was set up on the main branch requiring pull requests before merging and blocking force pushes. This was not straightforward on the first attempt — see the issues section — but is now confirmed active and working correctly: direct pushes are blocked, and only an authorised bypass can push automated updates.

5.3 Automated Daily Activity Digest

A system was built so that every day's work is documented automatically and consistently, without relying on anyone remembering to write it up by hand. It has two parts:

- Automated activity log — a script reads that day's git commits (grouped by folder, i.e. by track) and writes them into docs/day-reports/<date>.md. This runs on a schedule, whenever a pull request is merged, or on demand. It is generated straight from git history and cannot be skipped or forgotten.
- Manual team notes — below the automated log, each person who did work that day adds a short entry: what was done, what worked, what didn't, the output, and next steps. Git can only show what changed, not whether it actually worked — that part needs a person.

This system was audited using Claude Code before being trusted for daily use, which surfaced and fixed two real bugs (a date-boundary miscalculation and a git history edge case that could silently drop a commit from a report) — see Section 6. It is now confirmed working end-to-end on the live repository.

6. Issues Faced and How We Solved Them

Issue 1: The branch protection ruleset appeared to be set up, but wasn't actually enforced.

Solution: Rulesets do not enforce on a private repository on GitHub's free plan. Solved by making the repository public, after confirming there was nothing sensitive in it.

Issue 2: After properly enabling the "require pull request" rule, the automated daily digest's own push to main started getting rejected by that same rule.

Solution: The digest bot needs to push directly, but the rule blocks direct pushes for everyone by design. Solved by adding a bypass — discovered along the way that this org's plan does not offer "GitHub Actions" as a selectable bypass actor, so a personal access token belonging to a repository admin was used instead, stored as a repository secret. The push now correctly counts as an authorised admin bypass.

Issue 3: A pull request claiming to contain bug fixes was merged, but the actual file content on the main branch still had the original bugs.

Solution: An older, unfixed version of the files had been dropped into the local clone before committing, and the mismatch between the commit message and the real content wasn't caught before merging. Solved by re-auditing the live repository with Claude Code, which grepped the actual file content (not just the commit message) and confirmed the mismatch, then rewrote both files from verified, tested content and re-verified the diff before merging again.

Issue 4: Dashboard 2's new Left / Right / Backward / Start controls need command formats that don't exist yet in the shared schema.

Solution: Rather than guessing a format or leaving the buttons silently non-functional, this was flagged directly in the UI as pending, so it's a visible, tracked gap rather than a hidden one. To be resolved by agreeing the new command format and updating MESSAGE_SCHEMA.md before wiring the buttons live.

7. Result / Conclusion

A substantial amount landed today, across two different kinds of work:

- Backend — built from scratch and deployed (FastAPI + SQLite + WebSocket), with a full REST API, mock test scripts, and both dashboards successfully connected to it end-to-end.
- Both dashboards — extended with new features (History views, an expanded control layout on Dashboard 2) and re-pointed at the new backend instead of hardware directly.
- Repository governance — the GitHub organisation, branch protection, and the automated daily documentation system are all live, audited, and confirmed working.

ELM327 hands-on coding was deliberately deferred to reflect this morning's direction first. Nothing here blocks that work — it picks up cleanly next session.

8. Next Steps

- Confirm whether the self-hosted backend satisfies the "company cloud + RDBMS" direction, or whether it needs to move to company infrastructure once those details are available.
- Agree and add the new Dashboard 2 command formats (left/right/backward/start) to MESSAGE_SCHEMA.md, then wire those buttons live.
- Both dashboards to call the new history endpoints and build out their History views.
- Begin the in-house ELM327 wiring (direct to ESP32/CAN, no Bluetooth app) as directed this morning.
- Continue toward the Phase 1 proof-of-concept vehicle build, per this morning's guidance to prove the concept before attempting Ackermann steering.